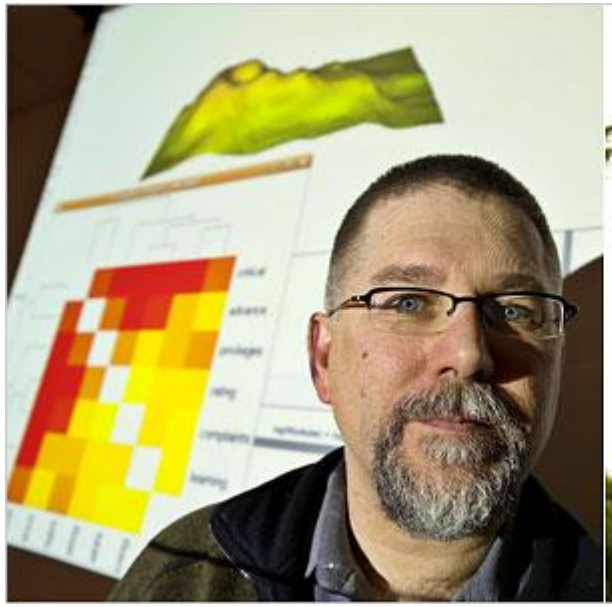
Intro to R





Robert Gentleman **Ross Ihaka**

Creators of R

Intro to R

- Go to https://cran.r-project.org/ and install version for your operating system
- Go to https://www.rstudio.com/products/rstudio/download/ and install R Studio IDE

You need to install R before installing R studio

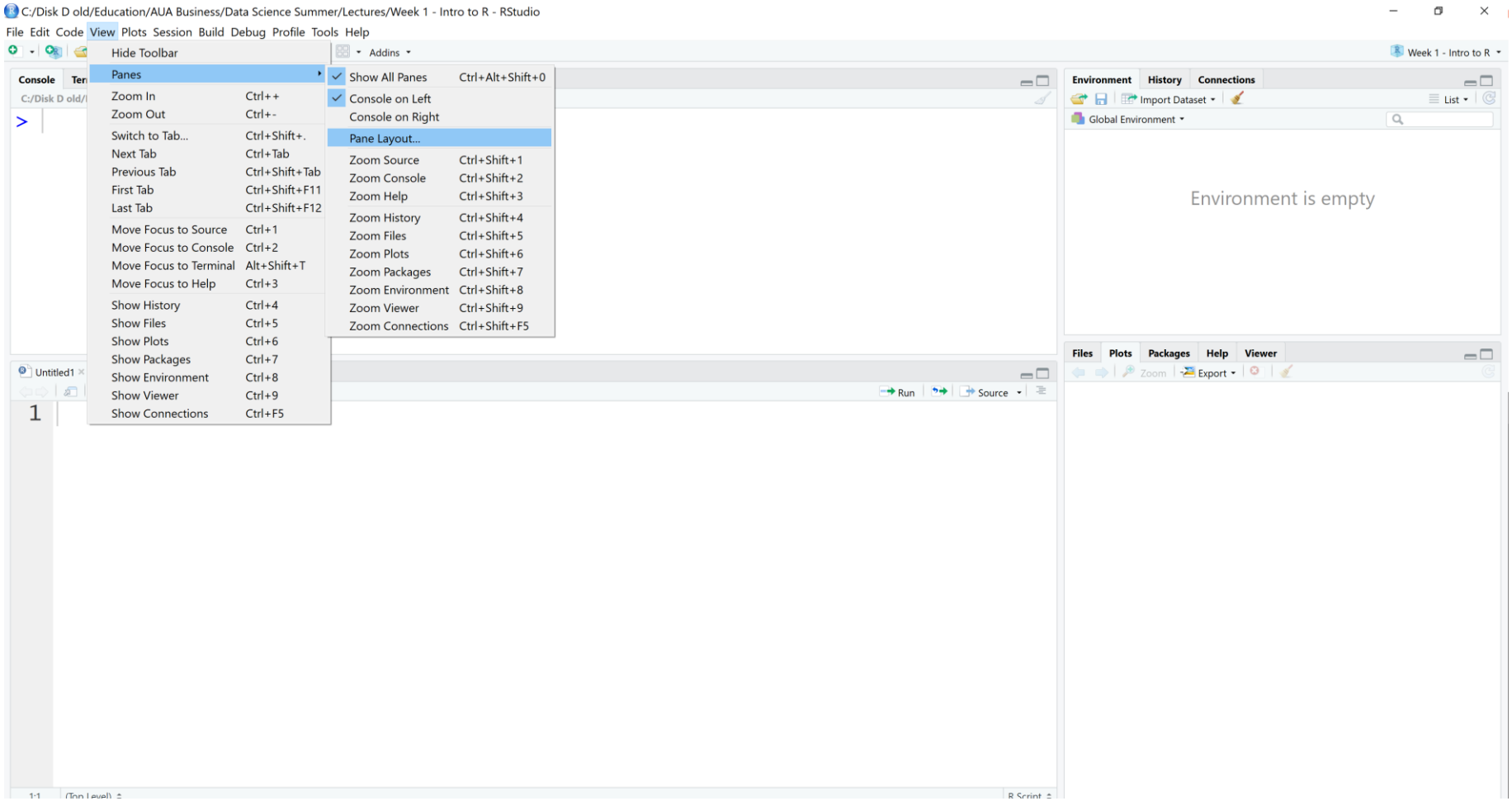
Alternatives

- Install Microsoft R https://mran.microsoft.com/open
- Use Microsoft Visual Studio (free version available)

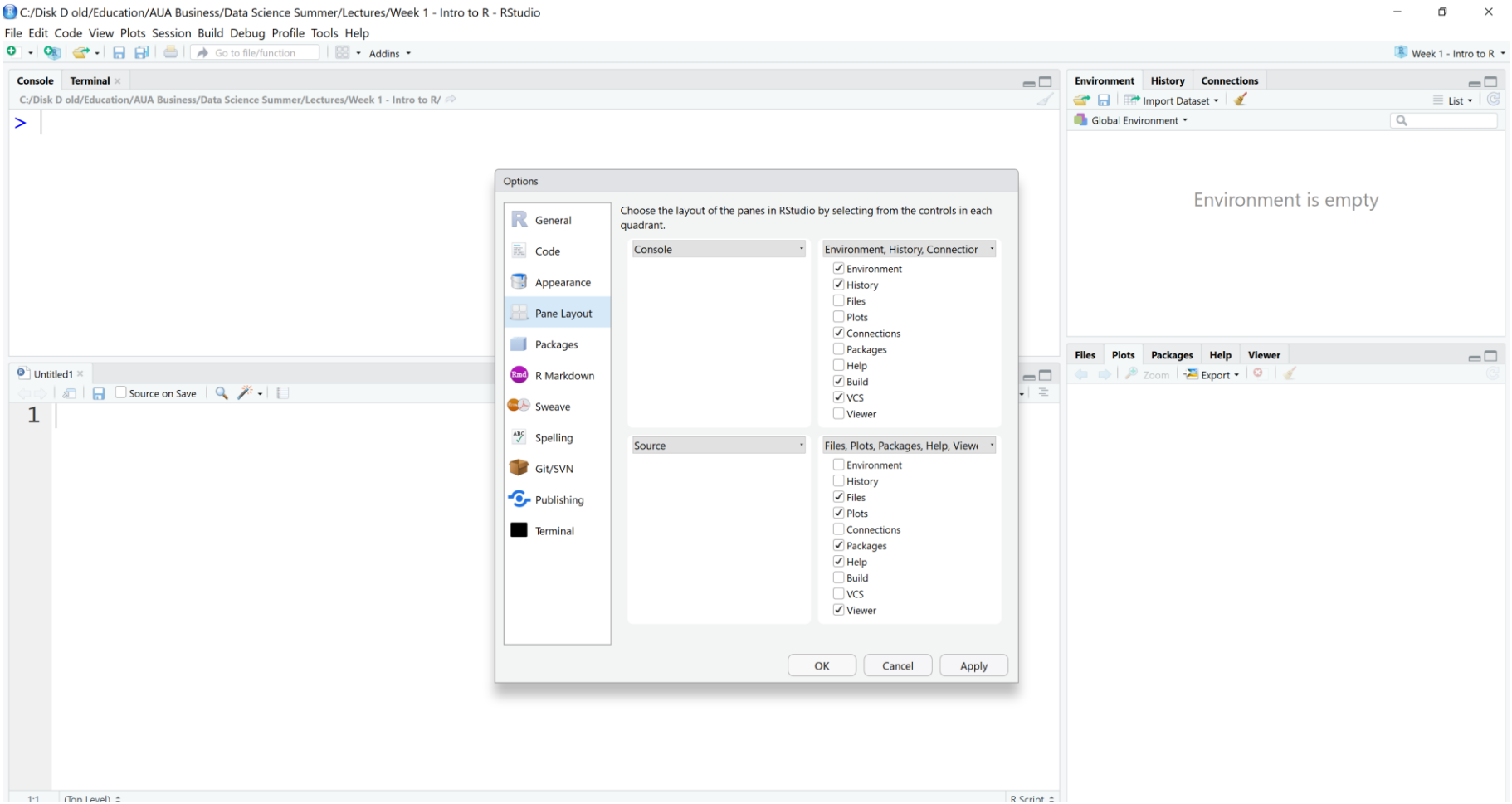


Intro to R

Configure the pane layout of R studio as you wish

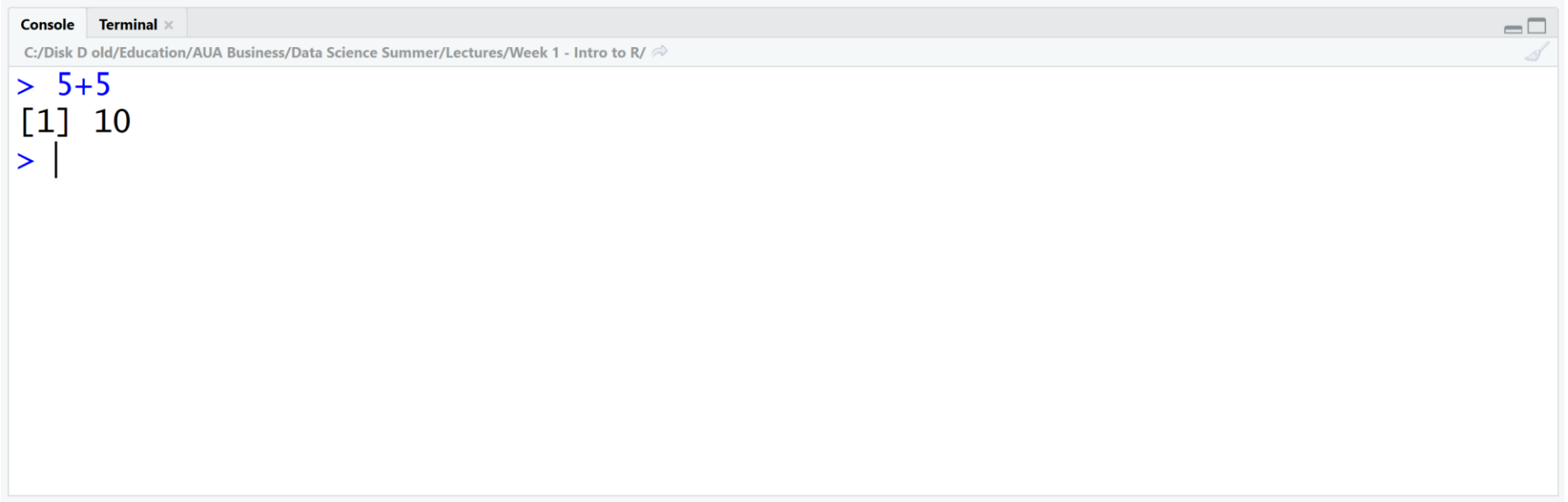


Intro to R


C:/Disk D old/Education/AUA Business/Data Science Summer/Lectures/Week 1 - Intro to R - RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Week 1 - Intro to R
Console
Terminal
C:/Disk D old/Education/AUA Business/Data Science Summer/Lectures/Week 1 - Intro to R/
Environment
History
Connections
Import Dataset
List
Global Environment
Environment is empty
Files
Plots
Packages
Help
Viewer
Zoom
Export
Untitled1
Source on Save
Options
General
Code
Appearance
Pane Layout
Packages
R Markdown
Sweave
Spelling
Git/SVN
Publishing
Terminal
Choose the layout of the panes in RStudio by selecting from the controls in each quadrant.
Console
Environment, History, Connectior
Environment
History
Files
Plots
Connections
Packages
Help
Build
VCS
Viewer
Source
Files, Plots, Packages, Help, Viewe
Environment
History
Files
Plots
Connections
Packages
Help
Build
VCS
Viewer
OK
Cancel
Apply
(Top Level)
R Script

Intro to R: console

Here you can type any valid R command after the > prompt followed by **Enter** and R will execute that command.



Use console as a calculator

[1] is the index for the output, just ignore it

Intro to R

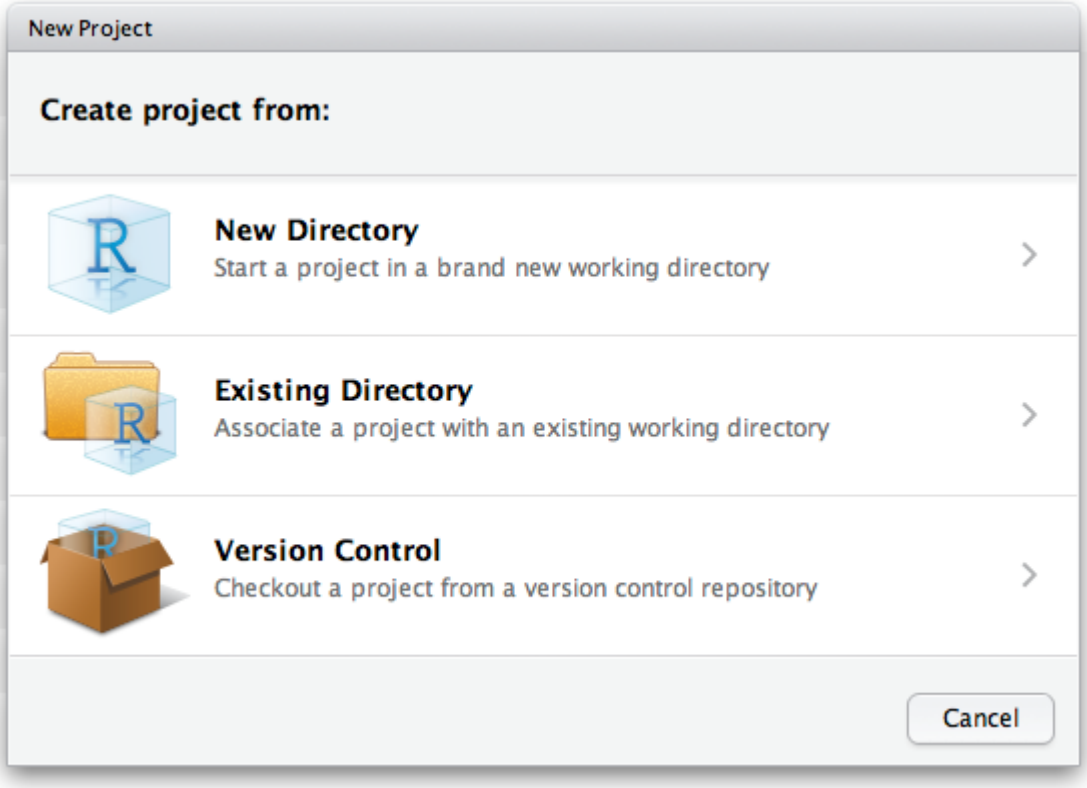
RStudio projects make it straightforward to divide your work into multiple contexts, each with their own working directory, workspace, history, and source documents.

Creating Projects

- RStudio projects are associated with R working directories. You can create an RStudio project:
  - In a brand new directory
  - In an existing directory where you already have R code and data
  - By cloning a version control (Git or Subversion) repository

Intro to R

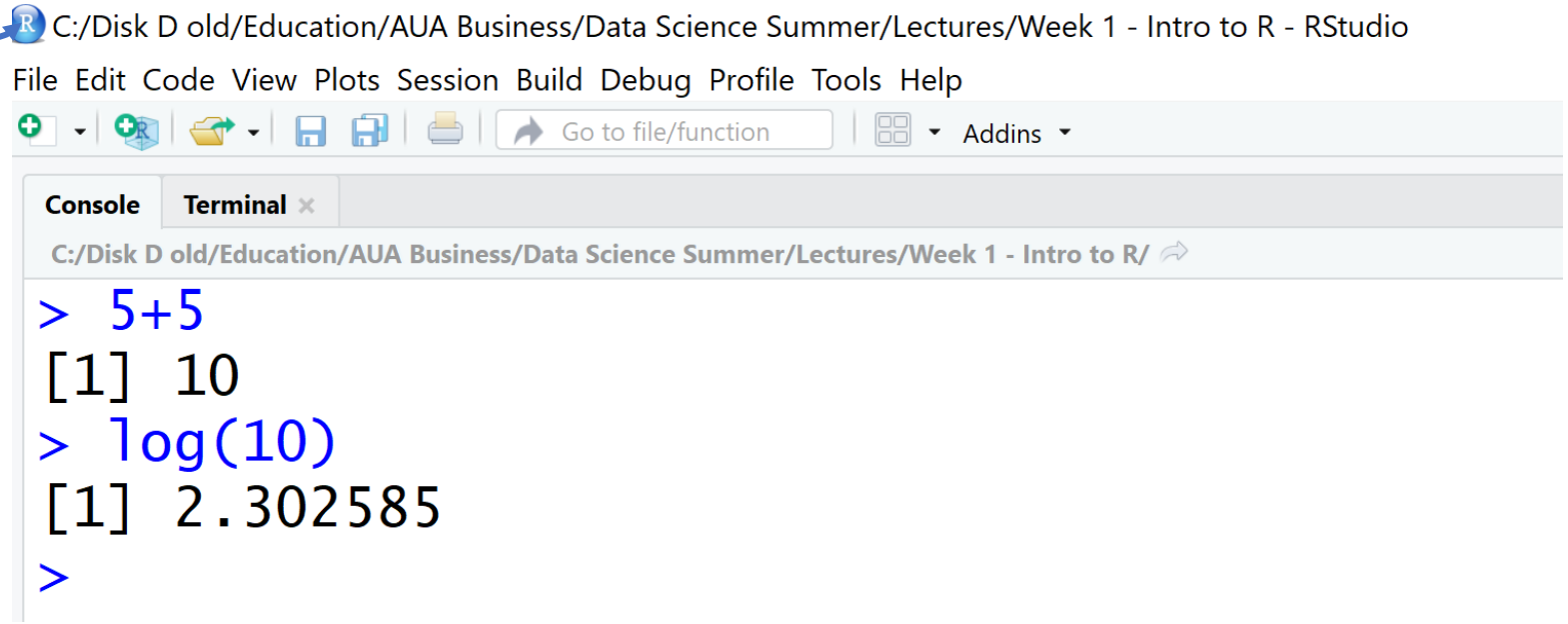
- Create an empty directory
- Go to File > New Project
- Chose existing directory



Intro to R

- Now when you have created your project, all files associated with the project need to be save in the project directory

- To check the project directory do getwd() in console



The name of the project appears at the top of R Studio window

Intro to R

When a new project is created RStudio:

- Creates a project file (with an .Rproj extension) within the project directory. This file contains various project options (discussed below) and can also be used as a shortcut for opening the project directly from the filesystem.

- Creates a hidden directory (named .Rproj.user) where project-specific temporary files (e.g. auto-saved source documents, window-state, etc.) are stored. This directory is also automatically added to .Rbuildignore, .gitignore, etc. if required.

- Loads the project into RStudio and display its name in the Projects toolbar (which is located on the far right side of the main toolbar)

Opening Projects

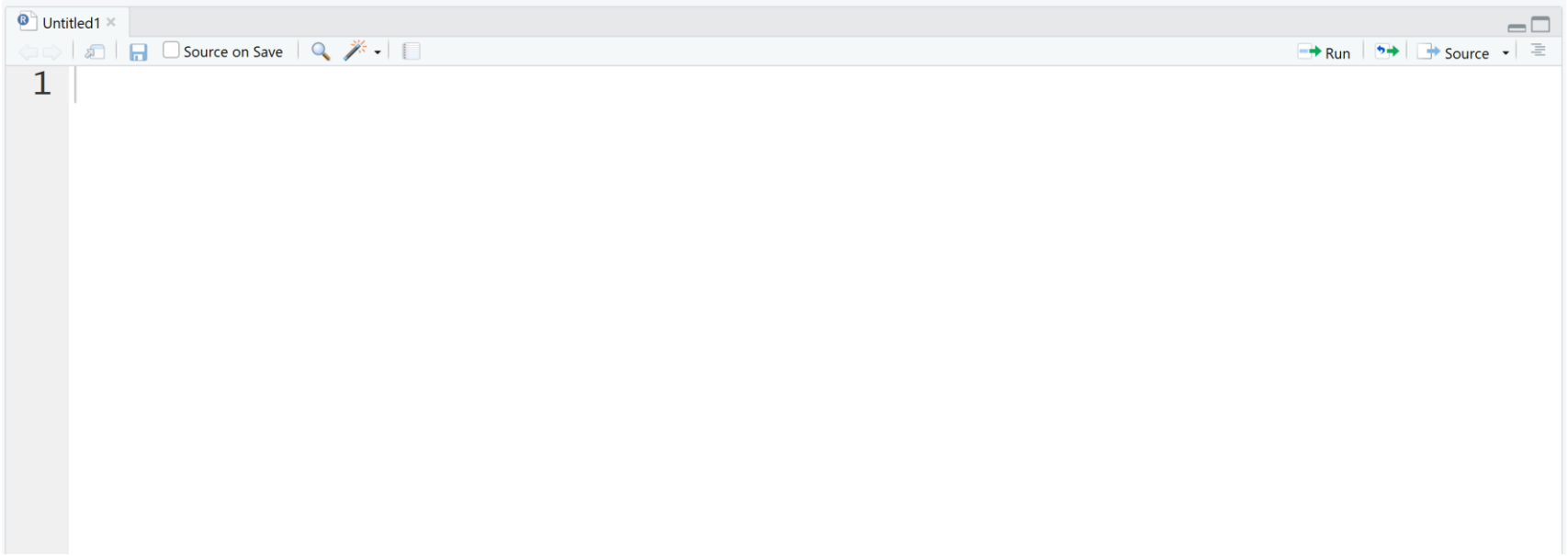
There are several ways to open a project:

- Using the **Open Project** command (available from both the Projects menu and the Projects toolbar) to browse for and select an existing project file (e.g. MyProject.Rproj).
- Selecting a project from the list of most recently opened projects (also available from both the Projects menu and toolbar).
- Double-clicking on the project file within the system shell (e.g. Windows Explorer, OSX Finder, etc.).

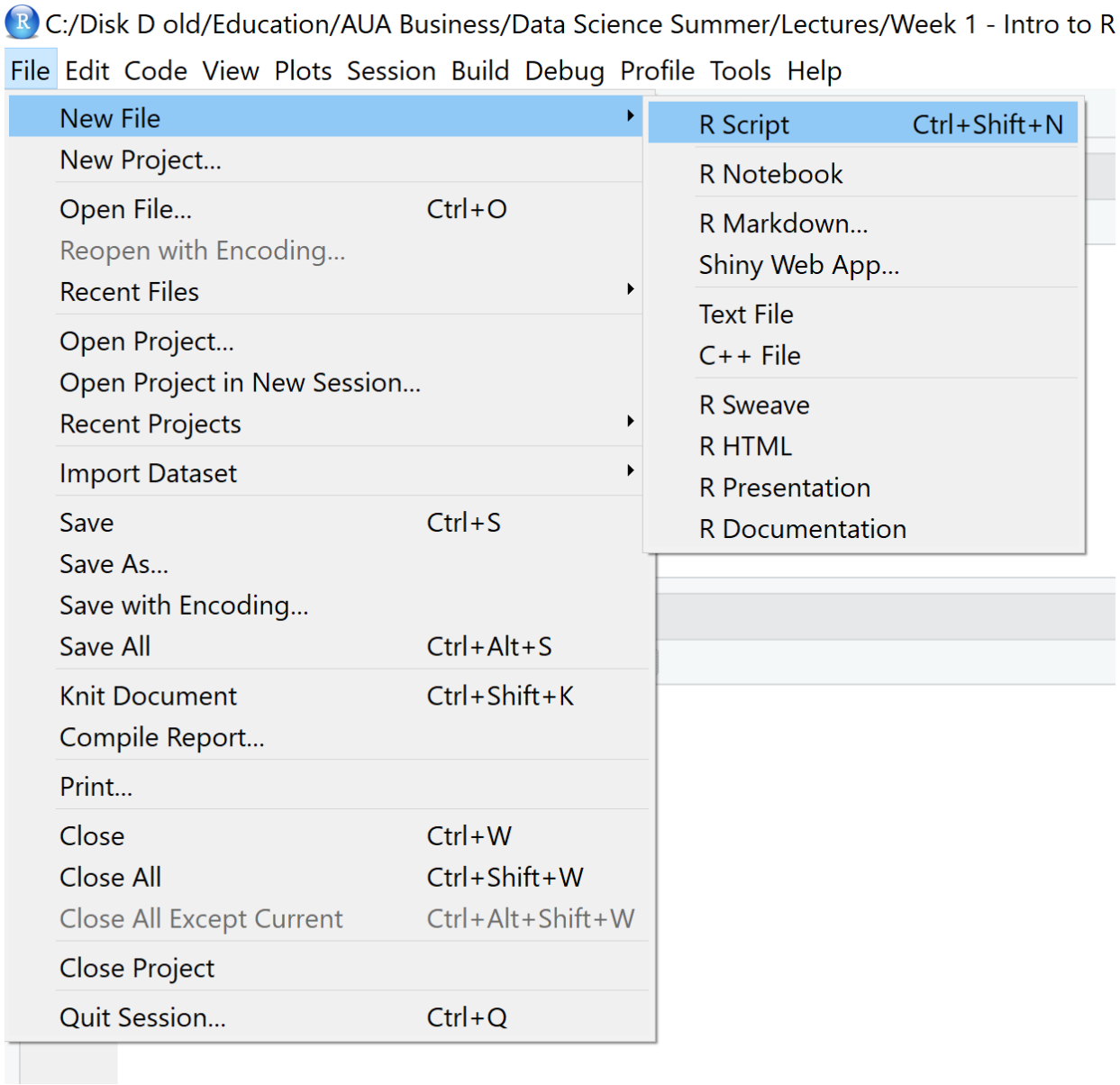
Intro to R

The source editor

- **If you plan to reuse your code, write it in source editor**



Intro to R

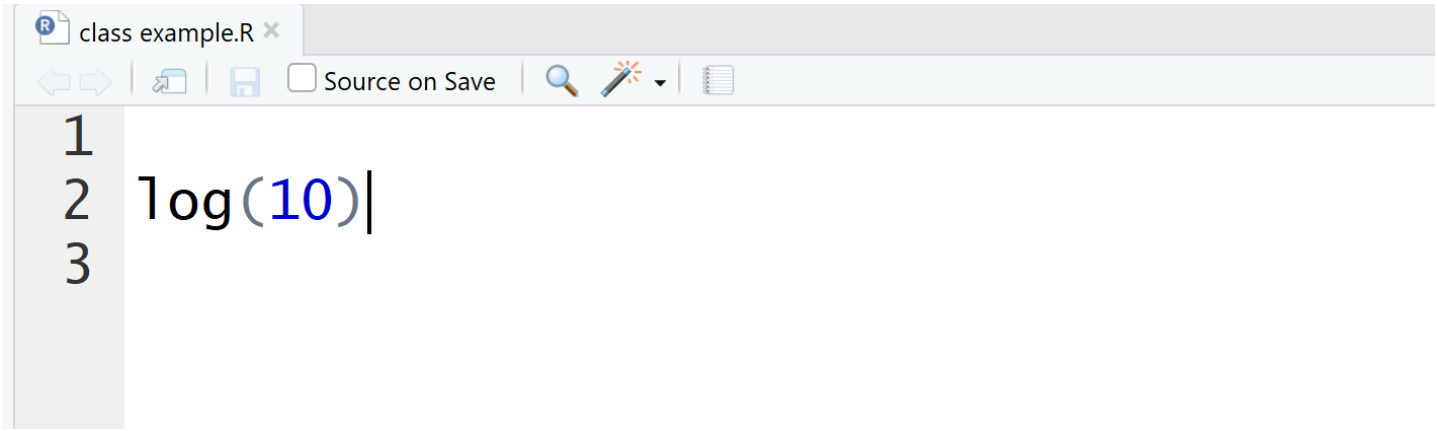


This will create new script file

Intro to R

- To run the script line from source editor, put the coursor anywhere on the line and hit **ctrl+enter**

- You will see the output in the console



Intro to R

- If you have a piece of text in your editor that is not a code (thus is not executable by R) then you need to comment it, add # before each line

- When run this line of code will printed in console as a text

```
C:/Disk D old/Education/AUA Business/Data Science Summer/Lectures/Week 1 - Intro to R/
> # This command will calculate the logarithm of 10
>
> log(10)
[1] 2.302585
>
> # This line will calculate factorial of 10
>
> factorial(10)
[1] 3628800
>
```

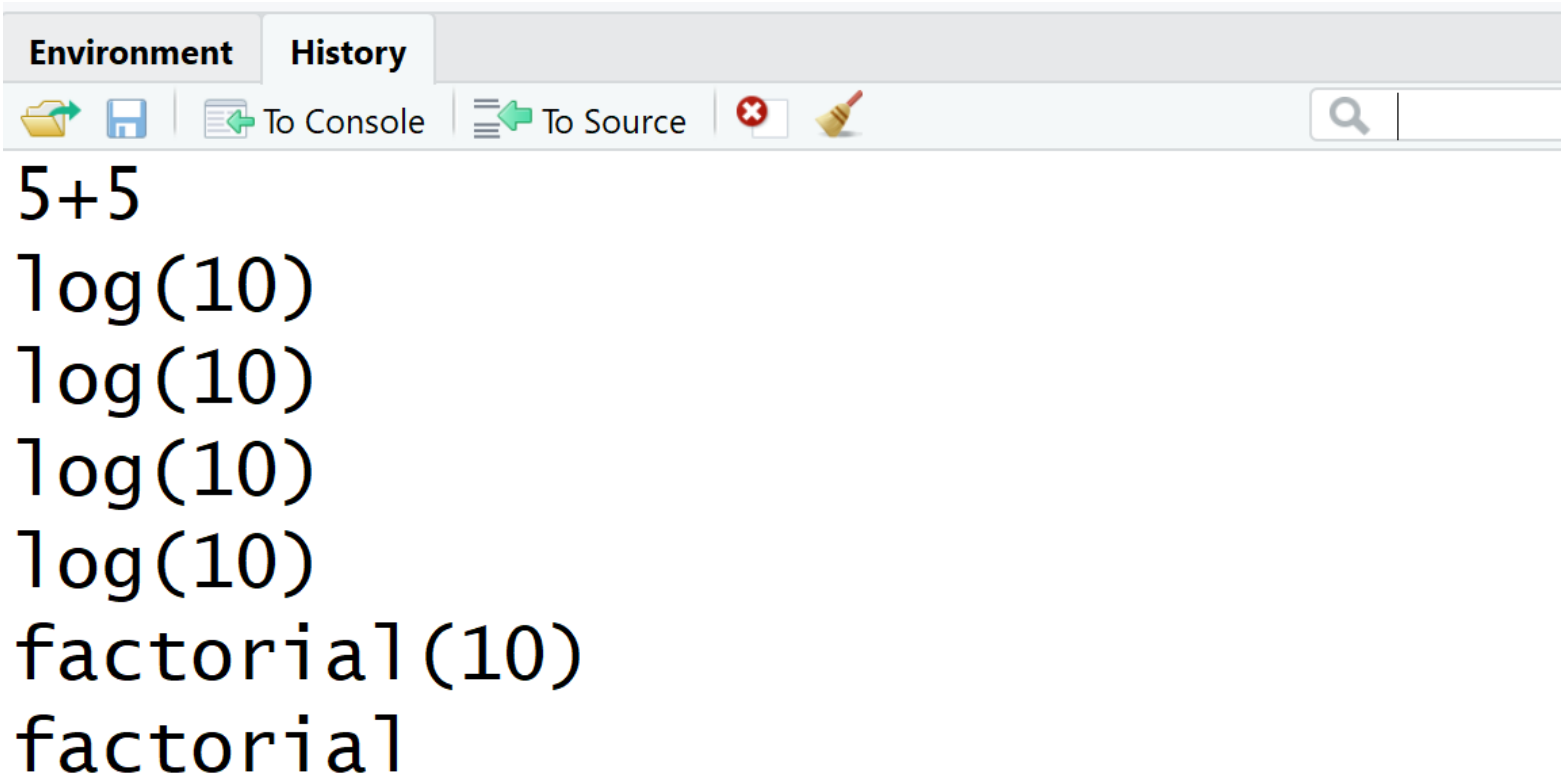
class example.R* | Source on Save

```
1
2 # This command will calculate the logarithm of 10
3
4 log(10)
5
6 # This line will calculate factorial of 10
7
8 factorial(10)
```

To comment large chunk of text, use ctrl+shift+c

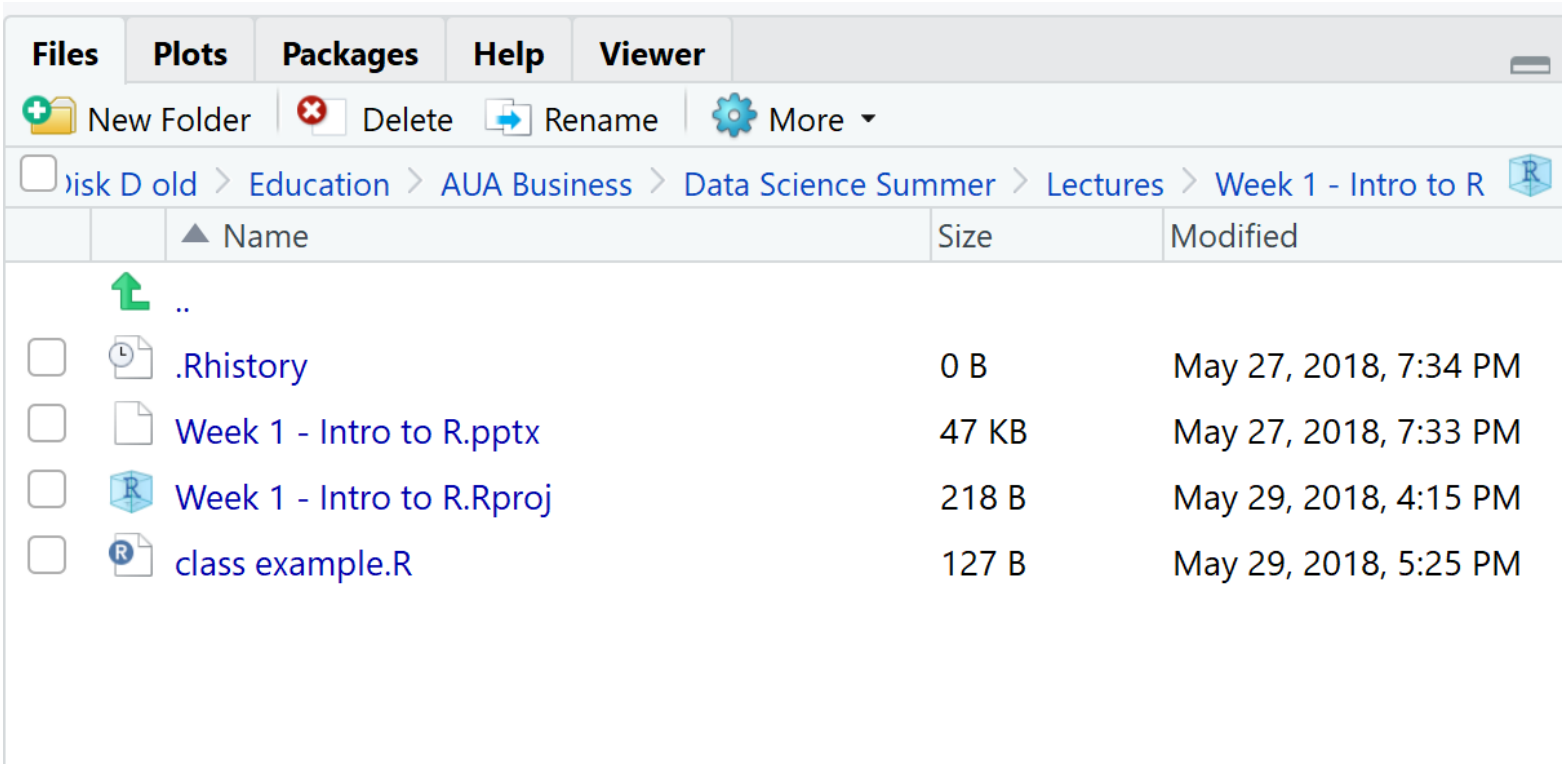
Intro to R

- The environment window contains objects (data, values, functions) R has currently stored in its memory.
- The history window shows all commands that were executed in the console.



Intro to R

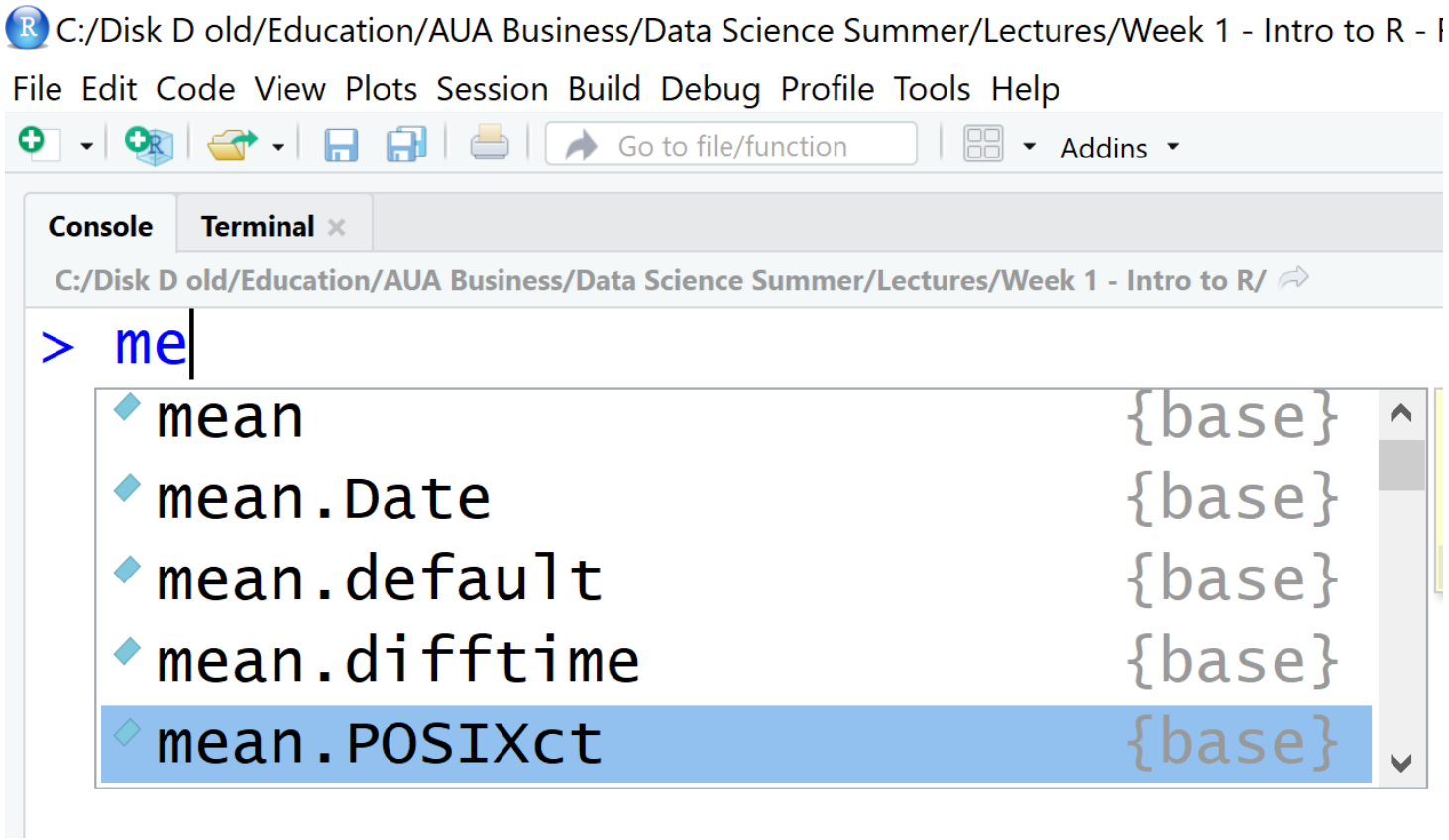
1.**Bottom right**: files, plots, packages, help, & viewer pane. Here you can open files, view plots, install and load packages, read man pages, and view markdown and other documents in the viewer tab.



Intro to R

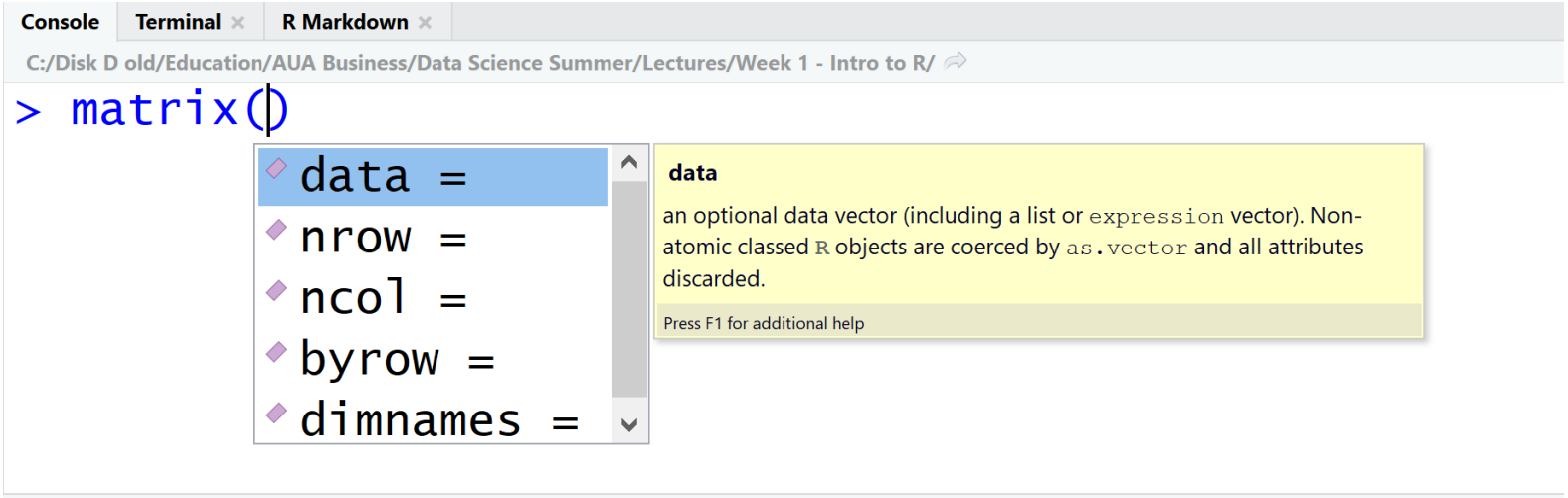
Some useful shortcuts:

- type first few letters of the function/object then hit **Tab** to open dropdown menu with possible options



Intro to R: Function

- to look at the function arguments, hit **Tab**
- you need to name the arguments if they are not in the same order as defined in within the function
- If the order is the same you can skip the names

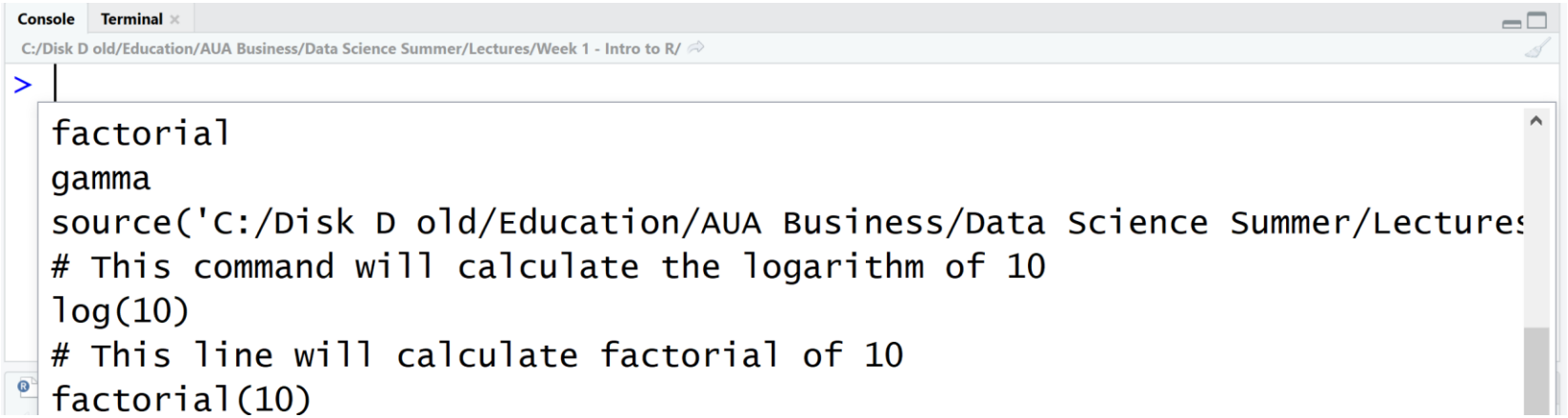


Intro to R

It's often the case that you want to re-execute commands that you previously entered. The RStudio console supports the ability to recall previous commands using the arrow keys:

- **Up** — Recall previous command(s)
- **Down** — Reverse of Up

You can even view a list of your recent commands by pressing Ctrl+Up on Windows or Command+Up on a Mac.



Intro to R

- When you download R from the Comprehensive R Archive Network (CRAN), you get that ``base'' R system
- The base R system comes with basic functionality; implements the R language
- One reason R is so useful is the large collection of packages that extend the basic functionality of R
- R packages are developed and published by the larger R community

Install packages

- Very often we are going to use functions that are not in base R, but instead are in external libraries (packages).
- R library is a collection of functions/datasets developed by the community members for specific purposes.
- You need to install library only once

```
install.packages("caret")
```

Install packages

- To use the functions from the library, you need to call it first.
- Need to do this only once per R session
- If you restarted R, you need to call it again.

```
library(caret)
createDataPartition()
```

If you want to use specific function from the library without calling the whole library, use **library::function()**

```
library(caret)
caret::createDataPartition()
```

Working with help

To access help/documentation on a function from R base package

```
?mean
```

```
## starting httpd help server ...
```

```
##  done
```

To access help/documentation on function from a library

```
??find.BIB
```

The same

```
?crossdes::find.BIB
```

Help on the package

```
help(package='ggplot2')
```

- Each package on CRAN has its own webpage
- This includes documentation
- And sometimes includes vignettes

If you have a specific task to do, then look at R Task View

- here are all the packages and R functionality described for the Time series analysis

https://cran.r-project.org/web/views/TimeSeries.html

R basic concepts

Intro to R: Data Structures

- Anything in R is an object
- Objects are assigned values using <- , an arrow formed out of < and -. (An equal sign, =, can also be used.) For example, the following command assigns the value 5 to the object x.
- R is case sensitive, thus Data10 and data10 are two different objects

```
x <- 5
x
```

```
## [1] 5
```

A tidy code requires a space before the assignment operator and a space after

- You can see what is inside the object just by simple entering the name of the object in command line
- When the object is created it should appear in your Environment Window
- If the object is not in your environment window you cannot work with it

Variable

A basic concept in (statistical) programming is called a **variable**.

A variable allows you to store a value (e.g. 4) or an object (e.g. a function description) in R. You can then later use this variable's name to easily access the value or the object that is stored within this variable.

You can assign a value 4 to a variable `my_var` with the command

my_var <-4

Variable

You can do basic operations with variables

```
my_var <- 4

my_var1 <- 5

my_var + my_var1
[1] 9
```

Or you can create a new variable

```
var2<- my_var+my_var1
```

Data types in R

R works with numerous data types. Some of the most basic types to get started are:

- Decimals values like 4.5 are called numerics.
- Natural numbers like 4 are called integers. Integers are also numerics.
- Boolean values (TRUE or FALSE) are called logical.
- Text (or string) values are called characters.

Note how the quotation marks indicate that "some text" is a character.

Checking data type

Assign integer 42 to variable my_numeric

String "universe“ to my_character

FALSE to boolean my_logical

Now run
class() for each variable

Vector

Vector – This data structures contain similar types of data, i.e., integer, double, logical, complex, etc. In order to create a vector in R Programming, c() function is used.

```
x <- c(10,5,6)
x
```

```
## [1] 10  5  6
```

This will create a vector with values from 1 to 10

```
x1 <- c(1:10)
x
```

```
## [1] 10  5  6
```

Check the class of the vector

```
class(x)
```

```
## [1] "numeric"
```

A character vector

```
y <- c("CS", "DS", "EC")
class(y)
```

```
## [1] "character"
```

Working with vectors

Types of variables: Vectors

Vectors are one-dimension arrays that can hold numeric data, character data, or logical data. In other words, a vector is a simple tool to store data.

In R, you create a vector with the combine function c(). You place the vector elements separated by a comma between the parentheses. For example:

```
numeric_vector <- c(1, 2, 3)

character_vector <- c("a", "b", "c")
```

Naming a vector

Create 2 new vectors

```
revenue <- c(140, 50, 20, 120, 240)
expenses <- c(24, 50, 100, 50, 10)
```

```
names(revenue)<-c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")
```

Subseting vectors

Subseting vector by element name

```
revenue["Monday"]
```

```
## Monday
##    140
```

Subseting element from vector by position. In R indexing starts from 1

```
revenue[4]
```

```
## Thursday
##      120
```

Subseting vectors

You can select 1 and 3 elements

```
revenue[c(1,3)]
```

```
##    Monday Wednesday
##       140         20
```

Or elements from 1 to 4

```
1:4
```

```
## [1] 1 2 3 4
```

```
revenue[c(1:4)]
```

```
##    Monday   Tuesday Wednesday  Thursday
##       140        50        20       120
```

Operations with vectors

Arithmetic operations of vectors are performed member-by-member, i.e., memberwise.

For example, the following three statements are completely equivalent:

```
c(1, 2, 3) + c(4, 5, 6)

c(1 + 4, 2 + 5, 3 + 6)

c(5, 7, 9)
```

You can also do the calculations with variables that represent vectors:

```
a <- c(1, 2, 3)
b <- c(4, 5, 6)
c <- a + b
```

Working with vectors

Find daily income

```
inc_d<-revenue-expenses
inc_d
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##       116         0       -80        70       230
```

Find the days with positive revenue

```
# This will give you a boolean vector
inc_d>0
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##      TRUE     FALSE     FALSE      TRUE      TRUE
```

Subsetting on condition

- Create a boolean vector with values TRUE when condition is met and FALSE when it does not
- From the initial object chose only elements where the value of Boolean vector is true

```
inc_pos<-inc_d[inc_d>0]
inc_pos
```

```
##    Monday Thursday    Friday
##       116       70       230
```

Logical comparison operators

The (logical) comparison operators known to R are:

< for less than

> for greater than

<= for less than or equal to

>= for greater than or equal to

== for equal to each other

!= not equal to each other

Difference between = and ==

- = is an assignment operator
- == is a comparison operato

```
x = 10
x == 10
```

```
## [1] TRUE
```

Working with vectors

```
#Find the non-negative income days
# find days when income was 0
```

You can make comparison between vectors two

```
revenue>expenses
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##      TRUE     FALSE     FALSE      TRUE      TRUE
```

Working with vectors

You can use sum() command to get the sum of all element in the vector

```
sum(inc_d)
```

```
## [1] 336
```

finding the average

```
mean(inc_d)
```

```
## [1] 67.2
```

you can do other operations with vectors too

```
# multiply by a number
inc_d*2
```

```
##    Monday   Tuesday Wednesday  Thursday    Friday
##       232         0      -160       140       460
```

Working with vectors: Recycling

If two vectors are of unequal length, the shorter one will be recycled in order to match the longer vector. For example, the following vectors u and v have different lengths, and their sum is computed by recycling values of the shorter vector u.

Recycling

```
u<-c(10,20,30)
v<-c(1,2,3,4,5,6,7,8,9)
u+v
```

```
## [1] 11 22 33 14 25 36 17 28 39
```

Working with vectors

dot product of the vectors

```
a<-c(1,4,5,3)
b<-c(5,3,9,8)
a%*%b
```

```
##      [,1]
## [1,]   86
```

Matrix multiplaction of two vectors

```
a%*%t(b)
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    5    3    9    8
## [2,]   20   12   36   32
## [3,]   25   15   45   40
## [4,]   15    9   27   24
```

Missing values in R

In R, missing values are represented by the symbol **NA** (not available). Impossible values (e.g., dividing by zero) are represented by the symbol **NaN** (not a number).

```
d<-c(0,1,6,NA,5 ,NaN)
```

```
is.na(d)
```

```
## [1] FALSE FALSE FALSE  TRUE FALSE  TRUE
```

```
!is.na(d)
```

```
## [1]  TRUE  TRUE  TRUE FALSE  TRUE FALSE
```

Operations with Matrices

Matrix

In R, a matrix is a collection of elements of the same data type (numeric, character, or logical) arranged into a fixed number of rows and columns. Since you are only working with rows and columns, a matrix is called two-dimensional.

You can construct a matrix in R with the matrix() function. Consider the following example:

Matrix

```
matrix(1:9, byrow = TRUE, nrow = 3)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

In the matrix() function:

- The first argument is the collection of elements that R will arrange into the rows and columns of the matrix. Here, we use 1:9 which is a shortcut for c(1, 2, 3, 4, 5, 6, 7, 8, 9).
- The argument byrow indicates that the matrix is filled by the rows. If we want the matrix to be filled by the columns, we just place byrow = FALSE.
- The third argument nrow indicates that the matrix should have three rows.

```
# Box office Star Wars (in millions!)
new_hope <- c(460.998, 314.4)
empire_strikes <- c(290.475, 247.900)
return_jedi <- c(309.306, 165.8)

# Create box_office
box_office <-c(new_hope, empire_strikes, return_jedi)
```

Construct star_wars_matrix

```
star_wars_matrix <- matrix(box_office, nrow = 3, byrow = TRUE)
star_wars_matrix
```

```
##          [,1]  [,2]
## [1,] 460.998 314.4
## [2,] 290.475 247.9
## [3,] 309.306 165.8
```

We could have done the same using rbind command

```
star_wars_m1<-rbind(new_hope, empire_strikes, return_jedi)
star_wars_m1
```

```
##                   [,1]  [,2]
## new_hope       460.998 314.4
## empire_strikes 290.475 247.9
## return_jedi    309.306 165.8
```

naming rows and columns

```
region <- c("US", "non-US")
titles <- c("A New Hope", "The Empire Strikes Back", "Return of the Jedi")
```

```
# Name the columns with region
colnames(star_wars_matrix) <- region

# Name the rows with titles
rownames(star_wars_matrix) <- titles

# Print out star_wars_matrix
star_wars_matrix
```

```
##                             US non-US
## A New Hope              460.998  314.4
## The Empire Strikes Back 290.475  247.9
## Return of the Jedi      309.306  165.8
```

Operations with matrices

Get US and Non-US box office for all tree movies combined

```
combined_vector<-colSums(star_wars_matrix)
combined_vector
```

```
##       US   non-US
## 1060.779  728.100
```

```
# take the first row -  A new hope
new_hope<-star_wars_matrix[1,]
new_hope
```

Subseting

Similar to vectors, you can use the square brackets [ ] to select one or multiple elements from a matrix. Whereas vectors have one dimension, matrices have two dimensions. You should therefore use a comma to separate that what to select from the rows from that what you want to select from the columns. For example:

- **my_matrix[1,2]** selects the element at the first row and second column.
- **my_matrix[1:3,2:4]** results in a matrix with the data on the rows 1, 2, 3 and columns 2, 3, 4.

If you want to select all elements of a row or a column, no number is needed before or after the comma, respectively:

- **my_matrix[,1]** selects all elements of the first column.
- **my_matrix[1,]** selects all elements of the first row.

Subseting matrices

```
# take the first row -  A new hope
new_hope<-star_wars_matrix[1,]
new_hope
```

```
##       US  non-US
## 460.998 314.400
```

```
# take the first column
US<-star_wars_matrix[,1]
```

```
#US box office for "A new hope"
star_wars_matrix[1,1]
```

Subsetting matrices

```
# You can do the subseting by the names of rows and columns as well,
# if they are specified
star_wars_matrix[,"non-US"]
```

```
##              A New Hope The Empire Strikes Back      Return of the Jedi
##                   314.4                   247.9                   165.8
```

```
star_wars_matrix["A New Hope","non-US"]
```

```
## [1] 314.4
```

Dataframes

Data frame

- Data frame is a data set that contain different data types instead of only one. You can combine numeric and character variables into the same dataset

- A data frame has the variables of a data set as columns and the observations as rows.

- This will be a familiar concept for those coming from different statistical software packages such as SAS or SPSS.

- Has the most use in Data analytics

Data frames – head and tail

- mtcars is an R built in dataset.
- to work with it start with **data(mtcars)**
- the dataframe will be visible as soon as you start working with it.

```
# look at the top 5 observations
head(mtcars, n=5)
```

```
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
```

Look at the bottom file observations

```
tail(mtcars, n=5)
```

```
##                mpg cyl  disp  hp drat    wt qsec vs am gear carb
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.9  1  1    5    2
## Ford Pantera L 15.8   8 351.0 264 4.22 3.170 14.5  0  1    5    4
## Ferrari Dino   19.7   6 145.0 175 3.62 2.770 15.5  0  1    5    6
## Maserati Bora  15.0   8 301.0 335 3.54 3.570 14.6  0  1    5    8
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.6  1  1    4    2
```

Structure of data frame

Another method that is often used to get a rapid overview of your data is the function `str()`. The function `str()` shows you the structure of your data set. For a data frame it tells you:

- The total number of observations (e.g. 32 car types)
- The total number of variables (e.g. 11 car features)
- A full list of the variables names (e.g. `mpg`, `cyl` ...)
- The data type of each variable (e.g. `num`)
- The first observations

Data frames: structure

```
str(mtcars)
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

Data frames, summary

```
summary(mtcars)
```

```
##       mpg             cyl             disp             hp       
##  Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
##  1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5  
##  Median :19.20   Median :6.000   Median :196.3   Median :123.0  
##  Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7  
##  3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0  
##  Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0  
##       drat             wt             qsec             vs        
##  Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000  
##  1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000  
##  Median :3.695   Median :3.325   Median :17.71   Median :0.0000  
##  Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375  
##  3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000  
##  Max.   :4.930   Max.   :5.424   Max.   :22.90   Max.   :1.0000  
##        am              gear            carb      
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000  
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000  
##  Median :0.0000   Median :4.000   Median :2.000  
##  Mean   :0.4062   Mean   :3.688   Mean   :2.812  
##  3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000  
##  Max.   :1.0000   Max.   :5.000   Max.   :8.000
```

Subseting dataframes

Similar to vectors and matrices, you select elements from a data frame with the help of square brackets `[ ]`. By using a comma, you can indicate what to select from the rows and the columns respectively. For example:

- `my_df[1,2]` selects the value at the first row and select element in `my_df`.
- `my_df[1:3,2:4]` selects rows 1, 2, 3 and columns 2, 3, 4 in `my_df`.

Sometimes you want to select all elements of a row or column. For example, `my_df[1,]` selects all elements of the first row. Let us now apply this technique on

Subseting dataframes

Subseting the dataframes

```
# get the dimensions of the dataframe
dim(mtcars)
```

```
## [1] 32 11
```

Create new dataframe with the first two columns and all the rows

```
df1<-mtcars[,1:2]
dim(df1)
```

```
## [1] 32  2
```

Create new dataframe with column 1,2,3 and 6, and first 10 rows

```
df2<-mtcars[1:10,c(1:3,6)]
dim(df2)
```

Sub setting data frames

Create new dataframe with all the variables, except 1 and 4

```
df3<-mtcars[,-c(1,4)]
dim(df3)
```

```
## [1] 32  9
```

if the columns or rows have names, you can subset by names as well

```
df4<-mtcars[,c("mpg", "gear", "disp")]
names(df4)
```

```
## [1] "mpg"  "gear" "disp"
```

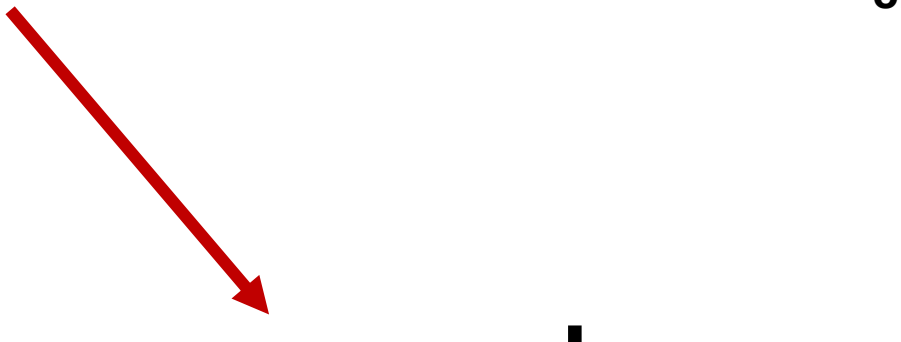
you can access a column in dataframe using $ sign if columns have names

```
mpg<-mtcars$mpg
mpg
```

```
##  [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2
## [15] 10.4 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4
## [29] 15.8 19.7 15.0 21.4
```

The name of the data frame or list

The name of the variable or object from the list

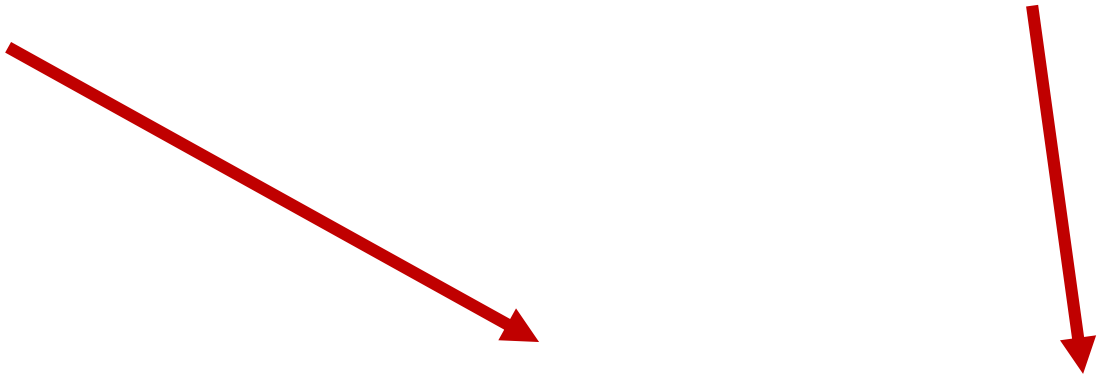


Data$Age

Intro to R: Accessing subset of the data frame

Extract cases (rows) from 1 to 500

Extract variables (columns) from 2 to 5



```
Data[1:500, 2:5]
```

Will return the value on first row and 4th column

```
dataframe[1 ,4 ]
```

Will return the value on first 10 rows and columns 4,5,6

```
dataframe[1:10 ,4:6 ]
```

Will return the first 10 rows and all columns

```
dataframe[1:10 , ]
```

If you want to select all elements for the given index, just leave the placeholder empty.

What will this command return ?

```
dataframe[,c(2,4:6) ]
```

Sub setting data frames

You can subset dataframe by rows by condition on any column

```
# keep only rows where mpg is more than or equal to 15
df5<-mtcars[mtcars$mpg>=15,]
summary(df5)
```

```
##       mpg             cyl             disp             hp
##  Min.   :15.00   Min.   :4.000   Min.   : 71.1   Min.   : 52.0
##  1st Qu.:17.55   1st Qu.:4.000   1st Qu.:120.2   1st Qu.: 94.0
##  Median :21.00   Median :6.000   Median :160.0   Median :110.0
##  Mean   :21.47   Mean   :5.852   Mean   :196.3   Mean   :131.6
##  3rd Qu.:23.60   3rd Qu.:8.000   3rd Qu.:275.8   3rd Qu.:175.0
##  Max.   :33.90   Max.   :8.000   Max.   :400.0   Max.   :335.0
##       drat             wt             qsec             vs
##  Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000
##  1st Qu.:3.115   1st Qu.:2.393   1st Qu.:16.96   1st Qu.:0.0000
##  Median :3.770   Median :3.170   Median :18.00   Median :1.0000
##  Mean   :3.666   Mean   :2.945   Mean   :18.03   Mean   :0.5185
##  3rd Qu.:4.000   3rd Qu.:3.450   3rd Qu.:19.17   3rd Qu.:1.0000
##  Max.   :4.930   Max.   :4.070   Max.   :22.90   Max.   :1.0000
##        am              gear            carb
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:1.500
##  Median :0.0000   Median :4.000   Median :2.000
##  Mean   :0.4815   Mean   :3.815   Mean   :2.593
##  3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:3.500
##  Max.   :1.0000   Max.   :5.000   Max.   :8.000
```

Sub setting data frames

Can subset by using more than 1 condition

```
# AND logical operator
df6<-mtcars[mtcars$mpg>15 & mtcars$carb<3,]
summary(df6)
```

```
##       mpg             cyl             disp             hp       
##  Min.   :15.20   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
##  1st Qu.:19.20   1st Qu.:4.000   1st Qu.: 95.1   1st Qu.: 66.0  
##  Median :22.80   Median :4.000   Median :121.0   Median : 97.0  
##  Mean   :23.61   Mean   :5.176   Mean   :177.7   Mean   :104.4  
##  3rd Qu.:27.30   3rd Qu.:6.000   3rd Qu.:258.0   3rd Qu.:113.0  
##  Max.   :33.90   Max.   :8.000   Max.   :400.0   Max.   :175.0  
##       drat             wt             qsec             vs        
##  Min.   :2.760   Min.   :1.513   Min.   :16.70   Min.   :0.0000  
##  1st Qu.:3.150   1st Qu.:2.140   1st Qu.:17.05   1st Qu.:0.0000  
##  Median :3.770   Median :2.780   Median :18.61   Median :1.0000  
##  Mean   :3.692   Mean   :2.709   Mean   :18.73   Mean   :0.7059  
##  3rd Qu.:4.080   3rd Qu.:3.435   3rd Qu.:19.90   3rd Qu.:1.0000  
##  Max.   :4.930   Max.   :3.845   Max.   :22.90   Max.   :1.0000  
##        am              gear            carb      
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000  
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:1.000  
##  Median :0.0000   Median :4.000   Median :2.000  
##  Mean   :0.4706   Mean   :3.706   Mean   :1.588  
##  3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:2.000  
##  Max.   :1.0000   Max.   :5.000   Max.   :2.000
```

Sub setting data frames

```
# The OR logical operator
df7<-mtcars[mtcars$mpg>15 | mtcars$carb<3,]
summary(df7)
```

```
##       mpg             cyl             disp             hp       
##  Min.   :15.20   Min.   :4.000   Min.   : 71.1   Min.   : 52.0  
##  1st Qu.:17.88   1st Qu.:4.000   1st Qu.:120.2   1st Qu.: 93.5  
##  Median :21.00   Median :6.000   Median :160.0   Median :110.0  
##  Mean   :21.72   Mean   :5.769   Mean   :192.3   Mean   :123.8  
##  3rd Qu.:24.00   3rd Qu.:8.000   3rd Qu.:275.8   3rd Qu.:168.8  
##  Max.   :33.90   Max.   :8.000   Max.   :400.0   Max.   :264.0  
##       drat             wt             qsec             vs        
##  Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000  
##  1st Qu.:3.098   1st Qu.:2.356   1st Qu.:17.02   1st Qu.:0.0000  
##  Median :3.810   Median :3.160   Median :18.15   Median :1.0000  
##  Mean   :3.671   Mean   :2.921   Mean   :18.16   Mean   :0.5385  
##  3rd Qu.:4.040   3rd Qu.:3.440   3rd Qu.:19.30   3rd Qu.:1.0000  
##  Max.   :4.930   Max.   :4.070   Max.   :22.90   Max.   :1.0000  
##        am              gear            carb      
##  Min.   :0.0000   Min.   :3.000   Min.   :1.000  
##  1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:1.250  
##  Median :0.0000   Median :4.000   Median :2.000  
##  Mean   :0.4615   Mean   :3.769   Mean   :2.385  
##  3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:3.000  
##  Max.   :1.0000   Max.   :5.000   Max.   :6.000
```

Data Frames: creating new data frame

Creating the final season schedule for Serie A.

The following varaibles describe the first 5 places for Serie A at 1996.

```
Team<-c("AC Milan","Juventus","Lazio","ACF Fiorentina","AS Roma","Parma AC")
Wins<-c(21,19,17,17,16,16)
Draws<-c(10,8,8,8,10,10)
Points<-c(73,65,59,59,58,58)
```

we can combine these variables into a dataframe

```
seriea<-data.frame(Team,Points, Wins, Draws)
class(seriea)
```

```
## [1] "data.frame"
```

Operations with dataframes

```
mean(seriea$Points)
```

```
## [1] 62
```

```
mean(seriea[,"Points"])
```

```
## [1] 62
```

We can create a new variable within the dataframe that will show the number of games each team has lost (total 34 games were played)

```
seriea$Lost<-34-(seriea$Wins+seriea$Draws)
```

- A data frame is the most common way of storing data in R, and if used systematically makes data analysis easier.

- Under the hood, a data frame is a list of equal-length vectors that can also have different types

```
df <- data.frame(x=1:10,y=11:20)
head(df, n=5)
```

```
##   x  y
## 1 1 11
## 2 2 12
## 3 3 13
## 4 4 14
## 5 5 15
```

Row numbers

Dataframe has column and row names

```
colnames(df)
```

```
## [1] "x" "y"
```

```
rownames(df)
```

```
##  [1] "1"  "2"  "3"  "4"  "5"  "6"  "7"  "8"  "9"  "10"
```

You can combine vectors with different type into a dataframe

First create a matrix with two vectors of different types

```
Club <- c("Juventus", "Napoli", "Roma", "Inter")
Points <- c(95,91,77,72)

df_seriea <- cbind(Club, Points)
str(df_seriea)
```

```
##  chr [1:4, 1:2] "Juventus" "Napoli" "Roma" "Inter" "95" "91" "77" "72"
##  - attr(*, "dimnames")=List of 2
##   ..$ : NULL
##   ..$ : chr [1:2] "Club" "Points"
```

```
class(df_seriea)
```

```
## [1] "matrix"
```

```
typeof(df_seriea)
```

```
## [1] "character"
```

Create a dataframe

```
df_seriea <- data.frame(Club, Points)
str(df_seriea)
```

```
## 'data.frame':    4 obs. of  2 variables:
##  $ Club  : Factor w/ 4 levels "Inter","Juventus",..: 2 3 4 1
##  $ Points: num  95 91 77 72
```

Two columns (vectors) in the df have different types

List

So far we have looked at the following objects

- **Vectors** (one dimensional array): can hold numeric, character or logical values. The elements in a vector all have the same data type.
- **Matrices** (two dimensional array): can hold numeric, character or logical values. The elements in a matrix all have the same data type.
- **Data frames** (two-dimensional objects): can hold numeric, character or logical values. Within a column all elements have the same data type, but different columns can be of different data type.

List

A list in R allows you to gather a variety of objects under one name (that is, the name of the list) in an ordered way. These objects can be matrices, vectors, data frames, even other lists, etc. It is not even required that these objects are related to each other in any way.

You could say that a list is some kind super data type: you can store practically any piece of information in it!

List

Let us create our first list! To construct a list you use the function `list()`:

```
my_list <- list(comp1, comp2 ...)
```

The arguments to the list function are the list components. Remember, these components can be matrices, vectors, other lists, ...

List

```r
# Vector with numerics from 1 up to 10
my_vector <- 1:10

# Matrix with numerics from 1 up to 9
my_matrix <- matrix(1:9, ncol = 3)

# First 5 elements of the built-in data frame mtcars
my_df <- mtcars[1:5,]

# a character
my_char<-"DataMata"

# combine into a list
my_list<-list(my_vector, my_matrix, my_df, my_char)
my_list
```

```
## [[1]]
##  [1]  1  2  3  4  5  6  7  8  9 10
##
## [[2]]
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
##
## [[3]]
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
##
## [[4]]
## [1] "DataMata"
```

List

You can see that elements in the list does not have names, which can be confusing. We need to name elements in the list

```
my_list <- list(name1 = your_comp1, name2 = your_comp2)
```

This creates a list with components that are named `name1`, `name2`, and so on. If you want to name your lists after you've created them, you can use the `names()` function as you did with vectors. The following commands are fully equivalent to the assignment above:

```
my_list <- list(your_comp1, your_comp2) names(my_list) <-
c("name1", "name2")
```

List

```
# Vector with numerics from 1 up to 10
my_vector <- 1:10

# Matrix with numerics from 1 up to 9
my_matrix <- matrix(1:9, ncol = 3)

# First 10 elements of the built-in data frame mtcars
my_df <- mtcars[1:5,]

# a character
my_char<-"DataMata"

# Adapt list() call to give the components names
my_list <- list(vec = my_vector, mat = my_matrix,
                df = my_df, char=my_char)

# Print out my_list
my_list
```

```
## $vec
##  [1]  1  2  3  4  5  6  7  8  9 10
##
## $mat
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
##
## $df
##                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
##
## $char
## [1] "DataMata"
```

List: The shining

New list for movie shining

```
moviename<-"The Shining"
actors<-c("Jack Nicholson","Shelley Duvall","Danny Lloyd",      "Scatman Crothers","Barry Nelson" )
reviews<-data.frame(scores=c(4.5,4,5),
        sources=c("IMDb1", "IMDb2", "IMDb3"),
        comments =c("Best Horror Film I Have Ever Seen",
    "A truly brilliant and scary film from Stanley Kubrick", "A masterpiece of psychological horror"))
shining_list<-list(moviename=moviename, actors=actors, reviews=reviews)
shining_list
```

```
## $moviename
## [1] "The Shining"
##
## $actors
## [1] "Jack Nicholson"   "Shelley Duvall"   "Danny Lloyd"
## [4] "Scatman Crothers" "Barry Nelson"
##
## $reviews
##   scores sources                                          comments
## 1    4.5   IMDb1                       Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                   A masterpiece of psychological horror
```

Sub setting the list

One way to select a component is using the numbered position of that component. For example, to "grab" the first component of shining_list you type

```
shining_list[[1]]
```

```
## [1] "The Shining"
```

Sub setting the list

A quick way to check this out is typing it in the console. Important to remember: to select elements from vectors, you use single square brackets: [ ]. Don't mix them up!

You can also refer to the names of the components, with [[ ]] or with the $ sign. Both will select the data frame representing the reviews:

```
shining_list[["reviews"]]
```

```
##   scores sources                                                comments
## 1    4.5   IMDb1                        Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                    A masterpiece of psychological horror
```

```
shining_list$reviews
```

```
##   scores sources                                                comments
## 1    4.5   IMDb1                        Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                    A masterpiece of psychological horror
```

Lists

Besides selecting components, you often need to select specific elements out of these components. For example, with shining_list[[2]][1] you select from the second component, actors (shining_list[[2]]), the first element ([1])

```
shining_list[[2]][1]
```

```
## [1] "Jack Nicholson"
```

Adding new value to the list

simply use c

```
shining_list<-c(shining_list, year=1980)
shining_list
```

```
## $moviename
## [1] "The Shining"
##
## $actors
## [1] "Jack Nicholson"   "Shelley Duvall"   "Danny Lloyd"
## [4] "Scatman Crothers" "Barry Nelson"
##
## $reviews
##   scores sources                                          comments
## 1    4.5   IMDb1                      Best Horror Film I Have Ever Seen
## 2    4.0   IMDb2 A truly brilliant and scary film from Stanley Kubrick
## 3    5.0   IMDb3                 A masterpiece of psychological horror
##
## $year
## [1] 1980
```

Data Types again

Data Types

Data types used in R

- Logical
- numeric
- factor
- character

Intro to R: Data types

Logical data type is one of the frequently used data type usually used for comparing two values. Values a logical data type takes is TRUE or FALSE.

```
a <- 10
a > 5
```

```
## [1] TRUE
```

```
log1 <- c(5, 6, TRUE)
typeof(log1)
```

```
## [1] "double"
```

```
log1
```

```
## [1] 5 6 1
```

Coercion of numeric and logical values will result as numeric

String literals or string values are stored as Character objects in R.

```
b <- c("Armenia", "Georgia", "Azerbaijan")
typeof(b)
```

```
## [1] "character"
```

```
b1 <- c(b, TRUE)
b1
```

```
## [1] "Armenia"    "Georgia"    "Azerbaijan" "TRUE"
```

Coercion

All elements of an atomic vector must be the same type, so when you attempt to combine different types they will be coerced to the most flexible type. Types from least to most flexible are: logical, integer, double, and character.

For example, combining a character and an integer yields a character:

```
x3 <- c("A",1)
x3
```

```
## [1] "A" "1"
```

```
class(x3)
```

```
## [1] "character"
```

Factors

- One important use of attributes is to define factors.
- A factor is a vector that can contain only predefined values, and is used to store categorical data.
- Factors are built on top of integer vectors using two attributes: the class, "factor", which makes them behave differently from regular integer vectors, and the levels, which defines the set of allowed values.

```
b <- as.factor(c("Armenia", "Georgia", "Azerbaijan"))
```

Factors in R are stored as a vector of integer values with a corresponding set of character values to use when the factor is displayed. The factor function is used to create a factor.

```
typeof(b)
```

```
## [1] "integer"
```

Look at the numeric values associated with factors

```
as.numeric(b)
```

```
## [1] 1 3 2
```

Look at the factor

```
b
```

```
## [1] Armenia    Georgia    Azerbaijan
## Levels: Armenia Azerbaijan Georgia
```

You can relevel the factor variable by changing the reference value

Relevel the variable

```
b<- relevel(b, ref='Azerbaijan')
levels(b)
```

```
## [1] "Azerbaijan" "Armenia"    "Georgia"
```

```
as.numeric(b)
```

```
## [1] 2 3 1
```

Functions

Functions

- In programming, you use functions to incorporate sets of instructions that you want to use repeatedly or that, because of their complexity, are better self-contained in a sub program and called when needed.
- A function is a piece of code written to carry out a specified task; it can or can not accept arguments or parameters and it can or can not return one or more values.
- In fact, there are several possible formal definitions of ‘function’ spanning from mathematics to computer science.
- **Generically, its arguments constitute the input and return values as output.**

Functions

- Functions have named arguments which potentially have default values.
- The formal arguments are the arguments included in the function definition.
- The formals function returns a list of all the formal arguments of a function.
- Not every function call in R makes use of all the formal arguments.
- Function arguments can be missing or might have default values

Functions

functions runif generate random numbers from continuous uniform distribution
Arguments are:

- n, number of observations, with no default so needs to be specified
- min, the minimum
- max, the maximum

```
formals(runif)
```

```
## $n
##
##
## $min
## [1] 0
##
## $max
## [1] 1
```

```
args(runif)
```

```
## function (n, min = 0, max = 1)
## NULL
```

Matching arguments:

R functions arguments can be matched positionally or by name. So the following calls to runif are all equivalent (you can mix them as well).

Matching by name

```
runif(min=1, n=10,max=2)
```

```
##  [1] 1.788855 1.073246 1.987114 1.170596 1.047545 1.641882 1.354573
##  [8] 1.841162 1.845753 1.177541
```

Matching by position

```
runif(10,1,2)
```

```
##  [1] 1.105142 1.024699 1.590241 1.502900 1.181286 1.082911 1.168374
##  [8] 1.053954 1.777667 1.686507
```

matched by position, this will give you an error

```
runif(1,10,2)
```

```
## Warning in runif(1, 10, 2): NAs produced
```

```
## [1] NaN
```

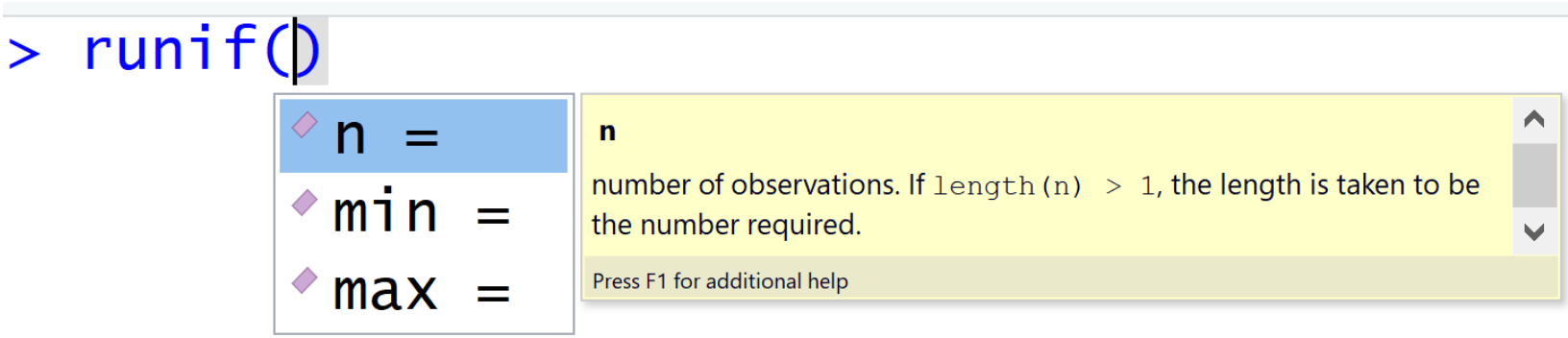
If the argument has a default value and is not defined then the default value is used.

Generated 10 numbers with min=0, max=1

```
runif(10)
```

```
##  [1] 0.47445298 0.08305635 0.45821497 0.66941373 0.31394192 0.96459659
##  [7] 0.60195719 0.43008084 0.43477999 0.44796031
```

You can look at the arguments of the function and their description by hitting ***tab*** inside the brackets



Reading data

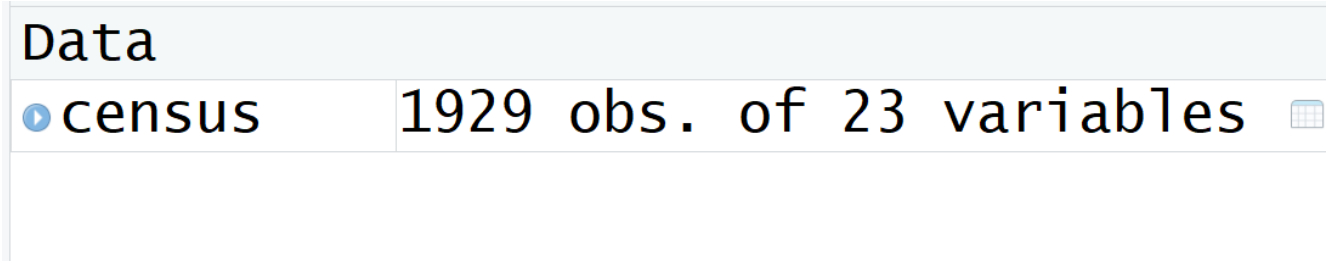
CSV files

csv (comma separated values)- the most common file format we will use

Use read.csv to get csv file into your R environment

```
census <- read.csv("census.csv")
```

- Look at the top right of R studio window, you need to see census object.
- If you don't see it there you cant work with it



Reading r data file

- R data files are saved with extension .rda
- Check to see if you have it in your project directory
- rm(list=ls()) will clean you environment
- To load rda file to environment, use load('filename.rda')
  - Pay attention: you are not assigning rda file to an object, just loading it

```
save(census, file='census.rda')
```

```
rm(list=ls())
```

```
load('census.rda')
```

Census R: factor and characters

```
summary(census)
```

```
##        id              race           region           sex       
##  Min.   :   1   Min.   :1.000   Min.   :1.000   Min.   :1.000  
##  1st Qu.: 524   1st Qu.:1.000   1st Qu.:2.000   1st Qu.:1.000  
##  Median :1015   Median :1.000   Median :4.000   Median :2.000  
##  Mean   :1019   Mean   :1.309   Mean   :4.085   Mean   :1.546  
##  3rd Qu.:1530   3rd Qu.:1.000   3rd Qu.:6.000   3rd Qu.:2.000  
##  Max.   :2023   Max.   :3.000   Max.   :8.000   Max.   :2.000  
##      degree           sibs           wrkstat         childs      
##  Min.   :0.000   Min.   : 0.000   Min.   :1.00   Min.   :0.000  
##  1st Qu.:1.000   1st Qu.: 2.000   1st Qu.:1.00   1st Qu.:0.000  
##  Median :1.000   Median : 3.000   Median :2.00   Median :2.000  
##  Mean   :1.565   Mean   : 3.775   Mean   :2.97   Mean   :1.955  
##  3rd Qu.:3.000   3rd Qu.: 5.000   3rd Qu.:5.00   3rd Qu.:3.000  
##  Max.   :8.000   Max.   :99.000   Max.   :8.00   Max.   :9.000  
##      hompop           earnrs          adults          educ       
##  Min.   : 1.000   Min.   :0.000   Min.   :1.000   Min.   : 0.00  
##  1st Qu.: 1.000   1st Qu.:1.000   1st Qu.:1.000   1st Qu.:12.00  
##  Median : 2.000   Median :1.000   Median :2.000   Median :13.00  
##  Mean   : 2.519   Mean   :1.453   Mean   :1.883   Mean   :13.58  
##  3rd Qu.: 3.000   3rd Qu.:2.000   3rd Qu.:2.000   3rd Qu.:16.00  
##  Max.   :11.000   Max.   :9.000   Max.   :9.000   Max.   :99.00  
##     marital          happy           income           age       
##  Min.   :1.000   Min.   :1.000   Min.   : 1.00   Min.   :18.00  
##  1st Qu.:1.000   1st Qu.:1.000   1st Qu.:11.00   1st Qu.:34.00  
##  Median :2.000   Median :2.000   Median :12.00   Median :47.00  
##  Mean   :2.515   Mean   :1.895   Mean   :14.57   Mean   :47.82  
##  3rd Qu.:5.000   3rd Qu.:2.000   3rd Qu.:12.00   3rd Qu.:60.00  
##  Max.   :5.000   Max.   :3.000   Max.   :98.00   Max.   :99.00
```

Census R: factor and characters

Variabel happy is a categorical variable where
1-Very happy
2-Pretty happy
3- Not to happy

We convert to factor specifying levels and labels

```
census$happy<-factor(census$happy, levels=c(1,2,3),
                     labels=c("Very Happy", "Pretty Happy", "Not too Happy"))
```

Census R: factor and characters

```
str(census)
```

```
## 'data.frame':    1929 obs. of  23 variables:
##  $ id      : int  1 2 3 4 6 11 12 14 17 19 ...
##  $ race    : int  3 3 2 2 2 2 2 2 1 1 ...
##  $ region  : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ sex     : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ degree  : int  1 4 1 1 3 4 4 1 1 3 ...
##  $ sibs    : int  0 1 1 2 3 4 8 7 3 0 ...
##  $ wrkstat : int  8 1 1 4 5 1 1 5 1 1 ...
##  $ childs  : int  5 2 1 1 2 2 0 4 0 0 ...
##  $ hompop  : int  2 1 2 3 2 2 1 2 2 1 ...
##  $ earnrs  : int  0 2 1 2 2 2 1 0 2 1 ...
##  $ adults  : int  1 1 2 3 2 2 1 2 2 1 ...
##  $ educ    : int  12 20 13 10 16 17 19 16 15 16 ...
##  $ marital : int  2 1 1 3 1 1 5 1 5 5 ...
##  $ happy   : Factor w/ 3 levels "Very Happy","Pretty Happy",..: 3 2 2 3 1 2 3 1 2 2 ...
##  $ income  : int  7 12 12 12 13 12 12 12 12 12 ...
##  $ age     : int  49 48 47 32 72 62 40 71 26 36 ...
```

type of the variable: factor

Census R: factor and characters

function table will count frequencies of each value in the variable

```
table(census$happy)
```

```
##
##     Very Happy  Pretty Happy Not too Happy
##            516           1099           314
```

Census R: factor and characters

- create another dataframe with with only region 8
- You can see that there are no very happy people here ☹
- So the count value for Very Happy is 0

```
census8 <- census[census$region==8,]
table(census8$happy)
```

```
##
##    Very Happy  Pretty Happy Not too Happy
##             0           178            54
```

Census R: factor and characters

- Now change the type of variable happy to character
- Subset it again
- count the frequencies
- Very happy is not even in the table

```
census$happy <- as.character(census$happy)
census8 <- census[census$region==8,]
table(census8$happy)
```

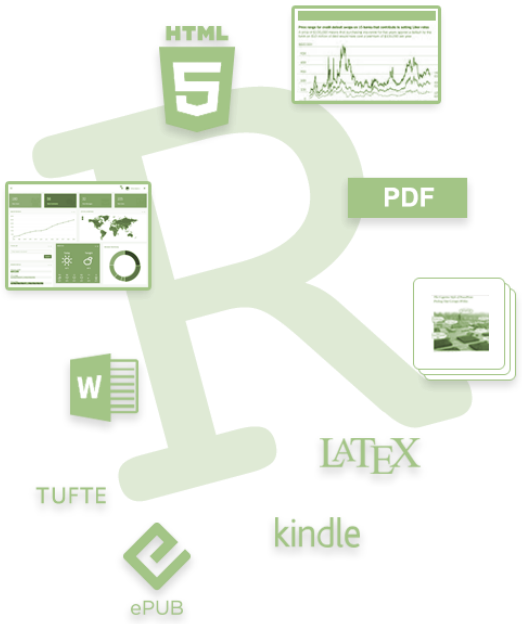
```
##
## Not too Happy  Pretty Happy
##            54           178
```

One reason why you need a factor() variable

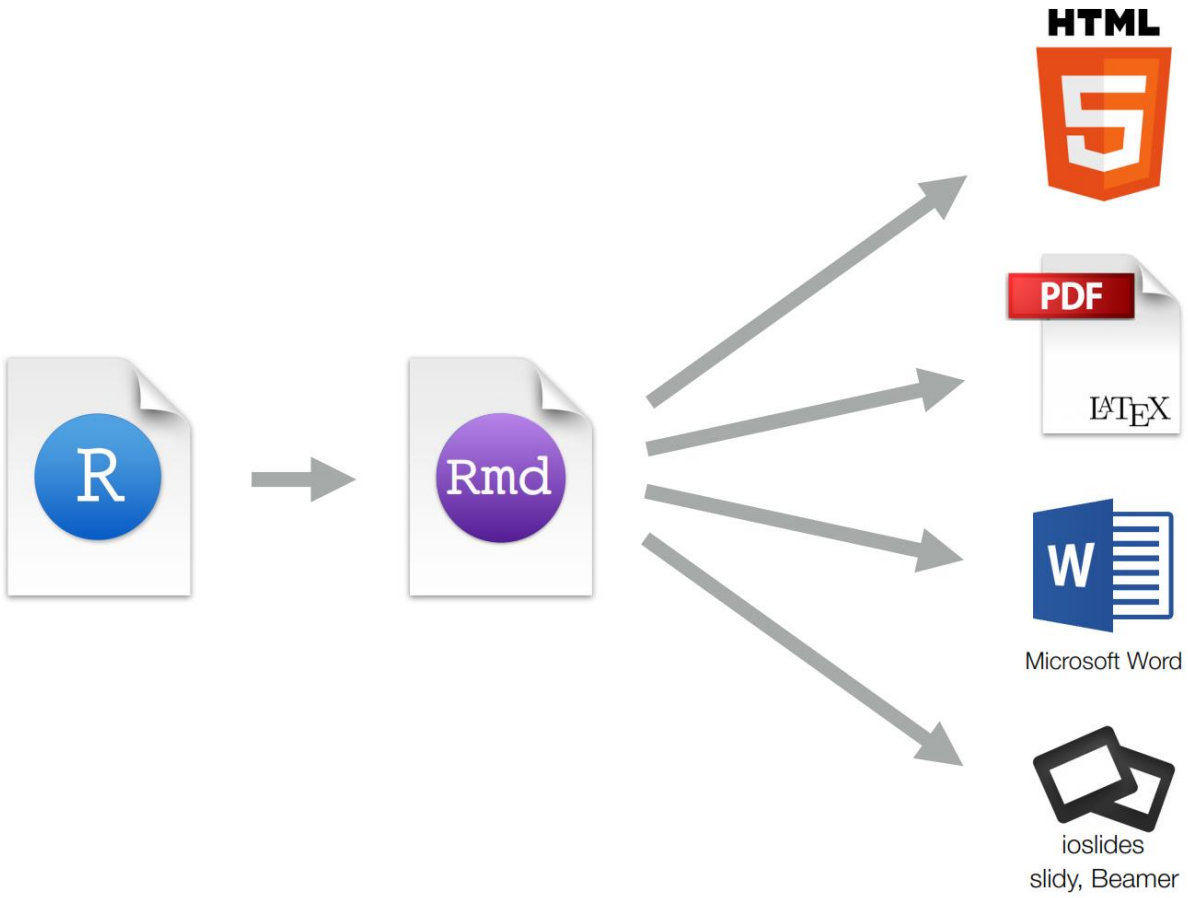
R markdown

R markdown

- R Markdown documents are fully reproducible
- It allows to combine text and code together in one report
- You can use multiple languages within markdown file (R, python, SQL)
- R Markdown supports dozens of static and dynamic output formats including HTML, PDF, MS Word, Beamer, HTML5 slides, etc
- R markdown is run in its own environment

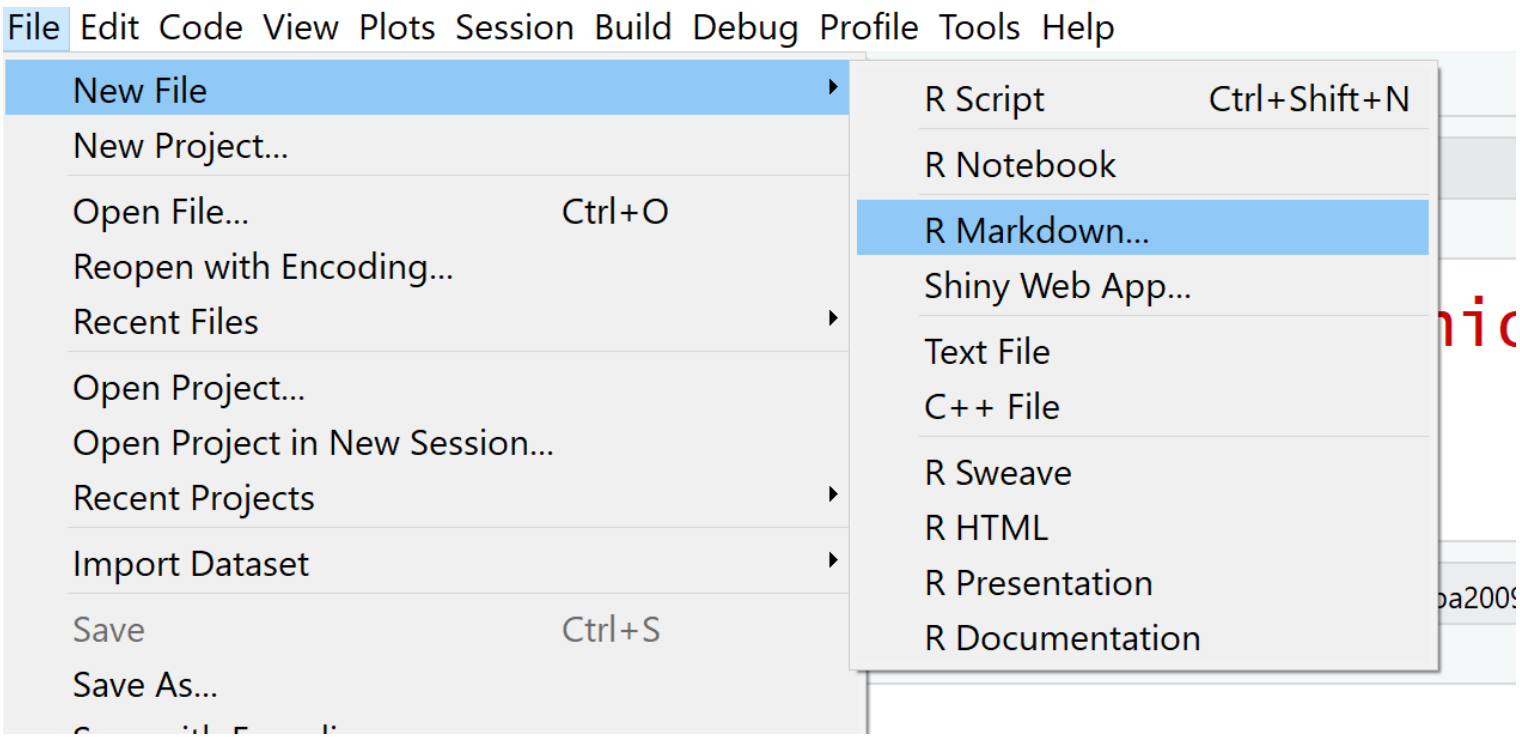


The workflow



R markdown

To open new markdown file



R markdown

The document below is a template R Markdown document. It includes the most familiar parts of an R Markdown document:

- A YAML header that contains some metadata **(1)**
- Narrative text written in Markdown **(2)**
- R code chunks surrounded by ```**{r}** and ```; a syntax that comes from the knitr package **(3)**

```
---
title: "Example"
author: "Habet Madoyan"
date: "June 16, 2018"
output: pdf_document
---

An R markdown document with some text and code for Data Science class

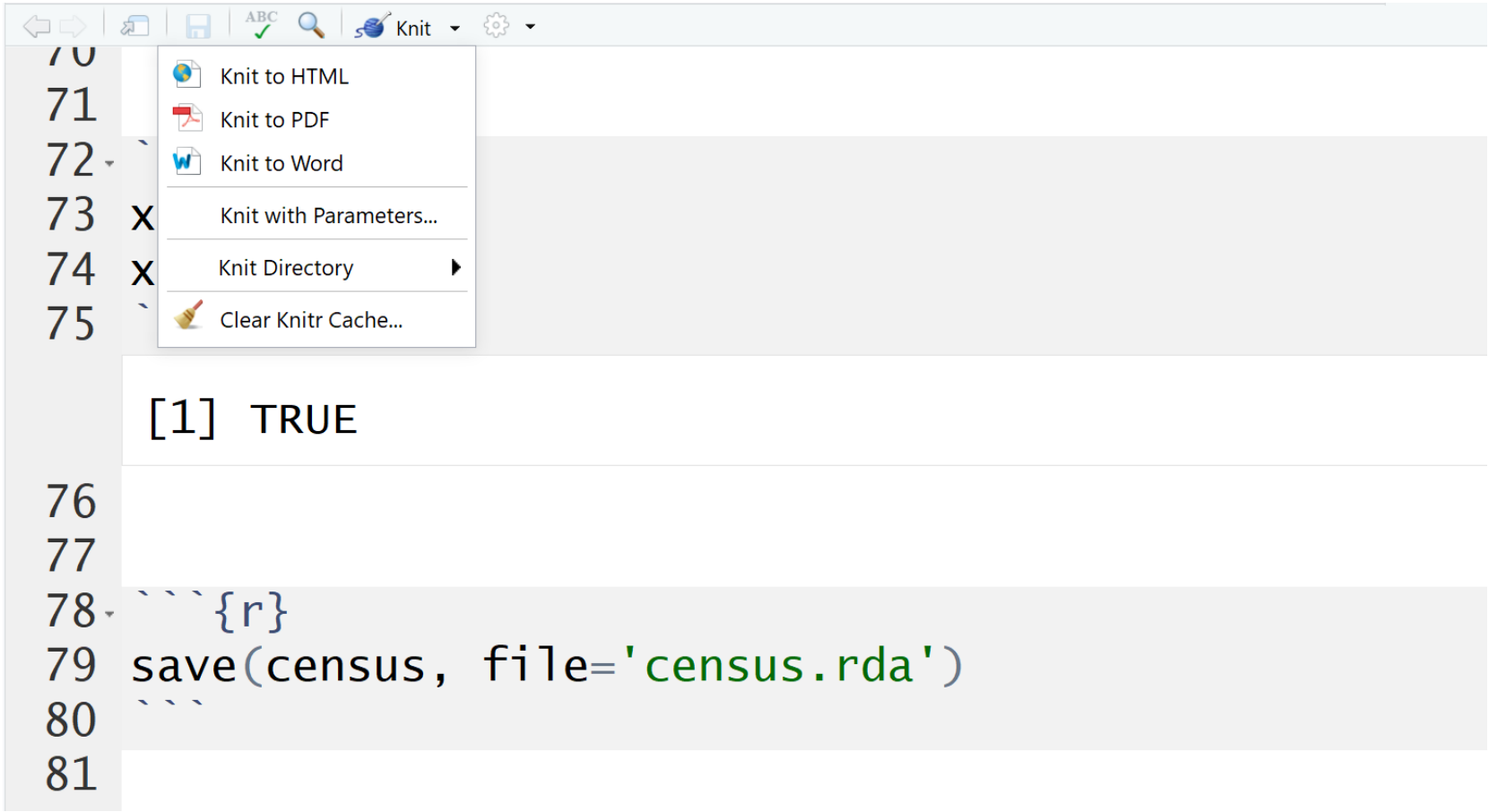

```{r}
head(mtcars, n=5)
```
```

1

2

3

- Now you can knit your report to different formats
- For pdf files, you need to have MikTex installed (it is going to take a lot of time)



Common errors

Common errors

- Check for spelling errors
- Check for typos
- Check for missed comma or quotation

Errors

When reading the data

```
> df <- read.csv("censas.csv")
Error in file(file, "rt") : cannot open the connection
In addition: Warning message:
In file(file, "rt") :
  cannot open file 'censas.csv': No such file or directory
>
```

- typo/spelling error – not *censas* but *census*
- if the name is right, than the file is not in your working directory

Could not find the function

- Typo, misspelling
- Forgot to call library that the given function belongs to

```
> qplot(df, Age, "histogram")
Error in qplot(df, Age, "histogram") : could not find function "qplot"
> 
```

- To use function qplot() from ggplot2 library, first call library(ggplot2).
- Remember need to call library only once per R session

- Subseting the mtcars dataframe where the variable drot>3
- No error is returned, however the resulting dataframe has 0 rows

```
> new_df <- mtcars[mtcars$drot > 3,]
> 
```

Why ?

Data	
new_df	0 obs. of 11 variables

Subseting the dataframe census by taking all the rows and columns 20 to 25

```
> census_df <- census[,20:25]
Error in `[.data.frame`(census, , 20:25) : undefined columns selected
> 
```

census has only 23 columns

While trying to install the library….
typo….

```
> install.packages("gglpot2")
Installing package into 'C:/Users/PC1/Documents/R/win-library/3.5'
(as 'lib' is unspecified)
Warning in install.packages :
  package 'gglpot2' is not available (for R version 3.5.0)
>
```

- Plus sign in the console and nothing is happening
- you forgot to close the parenthesis
- hit ESC while in console and fix the code

```
> mean(census$sibs
+
```

errors

- Some part of the code is in green,
- error indicators



```
44 library(ggplot2)
45 ggplot(dd, aes(x=season, y=NS, fill=league))+
46   geom_bar(stat="identity", position=position_dodge(width=1),
47   )+labs(x="Season", y="Noll-Skully Number",
48        title="Noll-Skully number for major european leagues,
49        fill="League")+
50   scale_fill_manual(values=wes_palettes$Darjeeling2)+
51   coord_cartesian(ylim=c(1,2.5))
52
53 ggsave("noll-skully.png")
54
55 scale_x_continuous(breaks=5)
56
57
58 library(SportsAnalytics270)
59 library(dplyr)
60 nfl2 <- nfl_st %>%
```

errors

- The strings need to be in quotation marks.
- Try to find the problem here

```
44 library(ggplot2)
45 ggplot(dd, aes(x=season, y=NS, fill=league))+
46   geom_bar(stat="identity", position=position_dodge(width=1),
47   )+labs(x="Season", y="Noll-Skully Number",
48        title="Noll-Skully number for major european leagues,
49        fill="League")+
50   scale_fill_manual(values=wes_palettes$Darjeeling2)+
51   coord_cartesian(ylim=c(1,2.5))
52
53 ggsave("noll-skully.png")
54
55 scale_x_continuous(breaks=5)
56
57
58 library(SportsAnalytics270)
59 library(dplyr)
60 nfl2 <- nfl_st %>%
```